

✓ TOÁN ỨNG DỤNG VÀ THỐNG KÊ

Đồ án Diagonalizable matrix

Mục lục

- [Thông tin sinh viên](#)
- [Mô tả bài toán và giải thuật chéo hoá](#)
- [Phần lập trình tự code](#)
- [Sử dụng thư viện numpy](#)
- [Sử dụng thư viện scipy](#)
- [So sánh kết quả và nhận xét chung](#)
- [Các ứng dụng của chéo hoá ma trận](#)
- [Triển khai ví dụ cho ứng dụng cụ thể](#)
- [Mô tả ý tưởng thực hiện và các hàm quan trọng](#)

=====

PHẦN THÔNG TIN SINH VIÊN

=====

Họ và tên: Nguyễn Hải Đăng

Mã số sinh viên: 23127165

Lớp: 23CLC03

✓ =====

PHẦN BÀI TOÁN VÀ GIẢI THUẬT

Mô tả bài toán

1. Cho A là ma trận có thể chéo hoá được. Sinh viên viết chương trình tìm ma trận chéo P , P^{-1} và ma trận đường chéo D , biết rằng $A = PDP^{-1}$
2. Mở rộng:
 - Tìm hiểu hàm/ phương thức (chéo hoá) tương ứng của các thư viện và thực hiện nó, so sánh kết quả.
 - Tìm hiểu và trình bày ứng dụng của chéo hoá

Lưu ý:

- Sinh viên phải sử dụng thuật toán chéo hóa đã được hướng dẫn trong phần lý thuyết và bài tập. Sinh viên không được dùng các hàm có sẵn của các thư viện để chéo hóa.
- Để tìm ma trận nghịch đảo P^{-1} sinh viên sử dụng thuật toán Gauss – Jordan đã được hướng dẫn trong phần lý thuyết và bài tập. Sinh viên không được dùng các hàm có sẵn của các thư viện để tìm ma trận nghịch đảo.

Thuật toán chéo hóa ma trận (Diagonalization Algorithm)

Ma trận A được gọi là chéo hóa được nếu tồn tại một ma trận khả nghịch P và một ma trận đường chéo D sao cho $A = PDP^{-1}$. Ma trận D chứa các giá trị riêng (eigenvalues) của A trên đường chéo chính, và các cột của ma trận P là các vector riêng (eigenvectors) tương ứng.

Các bước để chéo hóa một ma trận A :

1. Tìm các giá trị riêng (Eigenvalues):

- Để tìm các giá trị riêng λ của ma trận A , chúng ta giải phương trình đặc trưng: $\det(A - \lambda I) = 0$, trong đó I là ma trận đơn vị cùng kích thước với A .

2. Tìm các vector riêng (Eigenvectors) tương ứng:

- Với mỗi giá trị riêng λ_i tìm được, giải hệ phương trình $(A - \lambda_i I)v = 0$ để tìm không gian riêng (eigenspace) tương ứng.
- Các vector cơ sở của không gian riêng này chính là các vector riêng v_i tương ứng với λ_i .

3. Xây dựng ma trận P và D :

- Ma trận đường chéo D được tạo thành bằng cách đặt các giá trị riêng λ_i trên đường chéo chính, theo thứ tự tương ứng với các vector riêng.

- Ma trận P được tạo thành bằng cách ghép các vector riêng v_i thành các cột. Thứ tự của các cột trong P phải khớp với thứ tự của các giá trị riêng trong D .

4. Kiểm tra tính chéo hóa:

- Một ma trận A kích thước $n \times n$ là chéo hóa được nếu và chỉ nếu tổng số chiều của các không gian riêng bằng n , hoặc nói cách khác, có đủ n vector riêng độc lập tuyến tính để tạo thành ma trận P khả nghịch.

5. Tìm ma trận nghịch đảo P^{-1} :

- Sử dụng thuật toán Gauss-Jordan để tìm P^{-1} . Thuật toán này biến đổi ma trận $[P|I]$ thành $[I|P^{-1}]$ thông qua các phép biến đổi hàng sơ cấp.

Chương trình tìm ma trận chéo P , P^{-1} và ma trận đường chéo D

Thuật toán Gauss-Jordan để tìm ma trận nghịch đảo P^{-1}

Để tìm ma trận nghịch đảo của một ma trận vuông A , thuật toán Gauss-Jordan thực hiện các bước sau:

- 1. Ghép ma trận:** Tạo ma trận mở rộng $[A|I]$, trong đó I là ma trận đơn vị cùng kích thước với A .
- 2. Biến đổi hàng sơ cấp:** Áp dụng các phép biến đổi hàng sơ cấp (hoán đổi hàng, nhân một hàng với một số vô hướng khác 0, cộng một bội số của một hàng vào một hàng khác) để biến đổi phần A của ma trận mở rộng thành ma trận đơn vị I .
- 3. Kết quả:** Khi phần bên trái trở thành ma trận đơn vị I , thì phần bên phải sẽ là ma trận nghịch đảo A^{-1} , tức là $[I|A^{-1}]$. Nếu không thể biến đổi phần A thành I , thì ma trận A không khả nghịch.

✓ =====

PHẦN LẬP TRÌNH

=====

=====

Các câu của bài tập dùng để kiểm tra và chạy thử

```
test_matrices = {
    "a": [[-1, 3],
          [-2, 4]],
    "b": [[5, 2],
```

```

        [9, 2]],
    "c": [[1, -1, -1],
          [1, 3, 1],
          [-3, 1, -1]],
    "d": [[5, -1, 1],
          [-1, 2, -2],
          [1, -2, 2]],
    "e": [[1, 3, 3],
          [-3, -5, -3],
          [3, 3, 1]],
    "f": [[4, 0, -1],
          [0, 3, 0],
          [1, 0, 2]],
    "g": [[3, 4, -4],
          [-2, -1, 2],
          [-2, 0, 1]],
    "h": [[0, 0, -2],
          [1, 2, 1],
          [1, 0, 3]],
    "i": [[1, 0, 0],
          [1, 2, 0],
          [-3, 5, 2]],
    "j": [[4, 0, 1],
          [-2, 1, 0],
          [-2, 0, 1]]
}

```

```

import math
import numpy as np # Import numpy để sử dụng cho ma trận kích thước n > 3

# PHẦN NÀY DÀNH CHO CÁC HÀM TỰ XÂY DỰNG (dùng cho n <= 3)

def custom_det(matrix):
    """Tính định thức của ma trận vuông. (Sử dụng quy tắc Sarrus cho ma trận 2x2,
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    elif n == 2:
        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    elif n == 3:
        return (matrix[0][0] * matrix[1][1] * matrix[2][2] +
                matrix[0][1] * matrix[1][2] * matrix[2][0] +
                matrix[0][2] * matrix[1][0] * matrix[2][1] -
                matrix[0][2] * matrix[1][1] * matrix[2][0] -
                matrix[0][1] * matrix[1][0] * matrix[2][2] -
                matrix[0][0] * matrix[1][2] * matrix[2][1])
    else:
        det = 0
        for j in range(n):
            minor = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += ((-1)**j) * matrix[0][j] * custom_det(minor)
        return det

def custom_inverse(matrix):

```

```

"""Tìm ma trận nghịch đảo bằng thuật toán Gauss-Jordan."""
n = len(matrix)
if n == 0 or len(matrix[0]) != n:
    raise ValueError("Ma trận phải là ma trận vuông.")

# Tạo ma trận mở rộng [A|I]
augmented_matrix = [row[:] + [1 if i == j else 0 for j in range(n)] for i, row in enumerate(matrix)]

for i in range(n):
    pivot_row = i
    while pivot_row < n and abs(augmented_matrix[pivot_row][i]) < 1e-9:
        pivot_row += 1
    if pivot_row == n:
        return None

    augmented_matrix[i], augmented_matrix[pivot_row] = augmented_matrix[pivot_row], augmented_matrix[i]

    pivot_val = augmented_matrix[i][i]
    for j in range(2 * n):
        augmented_matrix[i][j] /= pivot_val

    for r in range(n):
        if r != i:
            factor = augmented_matrix[r][i]
            for j in range(2 * n):
                augmented_matrix[r][j] -= factor * augmented_matrix[i][j]

inverse_matrix = [row[n:] for row in augmented_matrix]
return inverse_matrix

def get_characteristic_polynomial_coeffs(matrix):
    """Tính các hệ số của đa thức đặc trưng (chỉ dành cho ma trận 2x2 hoặc 3x3, c
    n = len(matrix)
    if n == 2:
        a, b = matrix[0]
        c, d = matrix[1]
        return [1, -(a + d), (a * d - b * c)]
    elif n == 3:
        tr_A = sum(matrix[i][i] for i in range(n))

        c2_A = (matrix[0][0]*matrix[1][1] - matrix[0][1]*matrix[1][0]) + \
            (matrix[0][0]*matrix[2][2] - matrix[0][2]*matrix[2][0]) + \
            (matrix[1][1]*matrix[2][2] - matrix[1][2]*matrix[2][1])

        det_A = custom_det(matrix)

        return [1, -tr_A, c2_A, -det_A]
    else:
        raise ValueError("Hàm này chỉ hỗ trợ ma trận 2x2 và 3x3 để tìm đa thức đặ

def evaluate_polynomial(coeffs, x):
    """Đánh giá giá trị của đa thức tại x. coeffs = [a_n, ..., a_1, a_0]"""
    val = 0
    for i, coeff in enumerate(reversed(coeffs)):

```

```

    val += coeff * (x ** i)
    return val

def differentiate_polynomial(coeffs):
    """Tìm đạo hàm của đa thức. coeffs = [a_n, ..., a_1, a_0]"""
    if len(coeffs) <= 1:
        return [0]

    deriv_coeffs = []
    for i in range(len(coeffs) - 1):
        deriv_coeffs.append(coeffs[i] * (len(coeffs) - 1 - i))
    return deriv_coeffs

def newton_raphson(coeffs, initial_guess):
    """Tìm nghiệm gần đúng của đa thức bằng phương pháp Newton-Raphson."""
    tol = 1e-7
    max_iter = 1000
    x = initial_guess
    deriv_coeffs = differentiate_polynomial(coeffs)

    for _ in range(max_iter):
        f_x = evaluate_polynomial(coeffs, x)
        f_prime_x = evaluate_polynomial(deriv_coeffs, x)

        if abs(f_prime_x) < 1e-10: # Tránh chia cho 0 nếu đạo hàm rất nhỏ
            break

        x_new = x - f_x / f_prime_x
        if abs(x_new - x) < tol:
            return x_new
        x = x_new
    return x

def polynomial_division(dividend_coeffs, divisor_root):
    """Chia đa thức dividend_coeffs cho (x - divisor_root) bằng lược đồ Horner. T
    n = len(dividend_coeffs)
    if n == 0:
        return [0]

    quotient_coeffs = [0] * (n - 1)

    quotient_coeffs[0] = dividend_coeffs[0]
    for i in range(1, n - 1):
        quotient_coeffs[i] = dividend_coeffs[i] + divisor_root * quotient_coeffs[

    return quotient_coeffs

def solve_quadratic(coeffs):
    """Giải phương trình bậc hai  $ax^2 + bx + c = 0$ ."""
    tol = 1e-9
    a, b, c = coeffs

    if abs(a) < tol:
        if abs(b) < tol:

```

```

        return []
    return [-c / b]

delta = b**2 - 4*a*c
if delta >= -tol:
    if delta < 0:
        delta = 0
    sqrt_delta = math.sqrt(delta)
    if abs(a) < tol:
        if abs(b) < tol:
            return []
        return [-c / b]
    root1 = (-b + sqrt_delta) / (2*a)
    root2 = (-b - sqrt_delta) / (2*a)
    if abs(root1 - root2) < tol:
        return [root1]
    return sorted([root1, root2])
else:
    return []

def solve_cubic(coeffs):
    """Giải phương trình bậc ba  $ax^3 + bx^2 + cx + d = 0$ ."""
    tol = 1e-7
    a, b, c, d = coeffs

    if abs(a) < tol:
        list = [b, c, d]
        return solve_quadratic(list)

    roots = []

    first_root = newton_raphson(coeffs, 0.1)
    if first_root is None or abs(evaluate_polynomial(coeffs, first_root)) > tol:
        first_root = newton_raphson(coeffs, -0.1)
        if first_root is None or abs(evaluate_polynomial(coeffs, first_root)) > tol:
            print("Cảnh báo: Không thể tìm thấy nghiệm thực bằng Newton-Raphson.")
            return []

    if first_root is not None:
        roots.append(first_root)

        q2, q1, q0 = polynomial_division(coeffs, first_root)

        list = [q2, q1, q0]
        remaining_roots = solve_quadratic(list)
        roots.extend(remaining_roots)

    unique_roots = []
    for r in roots:
        is_new = True
        for ur in unique_roots:
            if abs(r - ur) < tol:
                is_new = False
                break
        if is_new:

```

```

        unique_roots.append(r)

    return sorted([round(r, int(-math.log10(tol)) + 2) for r in unique_roots])

def row_echelon_form(matrix):
    """Đưa ma trận về dạng hàng rút gọn (Reduced Row Echelon Form – RREF) bằng kh
    tol = 1e-7
    m = [row[:] for row in matrix]
    rows = len(m)
    cols = len(m[0])

    lead = 0
    for r in range(rows):
        if lead >= cols:
            break
        i = r
        while i < rows and abs(m[i][lead]) < tol:
            i += 1
        if i != rows:
            m[r], m[i] = m[i], m[r]

            lv = m[r][lead]
            for j in range(cols):
                m[r][j] /= lv

            for i_row in range(rows):
                if i_row != r:
                    lv = m[i_row][lead]
                    for j in range(cols):
                        m[i_row][j] -= lv * m[r][j]

            lead += 1
    return m

def get_null_space_basis(matrix):
    tol = 1e-7
    n_rows = len(matrix)
    n_cols = len(matrix[0])

    rref_matrix = row_echelon_form(matrix)

    pivot_cols = []
    free_cols = []

    current_pivot_row = 0
    for j in range(n_cols):
        is_pivot = False
        if current_pivot_row < n_rows and abs(rref_matrix[current_pivot_row][j] -
            all_other_zeros = True
            for r_check in range(n_rows):
                if r_check != current_pivot_row and abs(rref_matrix[r_check][j])
                    all_other_zeros = False
                    break
            if all_other_zeros:
                pivot_cols.append(j)

```

```

        current_pivot_row += 1
        is_pivot = True
    if not is_pivot:
        free_cols.append(j)

if not free_cols:
    return []

basis_vectors = []
for free_col_idx in free_cols:
    basis_vector = [0.0] * n_cols
    basis_vector[free_col_idx] = 1.0

    for r_idx in range(n_rows):
        pivot_val_col_idx = -1
        for c_idx in pivot_cols:
            if abs(rref_matrix[r_idx][c_idx] - 1.0) < tol:
                pivot_val_col_idx = c_idx
                break

        if pivot_val_col_idx != -1:
            sum_free_terms = 0
            for fc_idx in free_cols:
                sum_free_terms += rref_matrix[r_idx][fc_idx] * basis_vector[fc_idx]

            basis_vector[pivot_val_col_idx] = -sum_free_terms

    basis_vectors.append(basis_vector)

return basis_vectors

def multiply_matrices(A, B):
    """Nhân hai ma trận A và B. Trả về ma trận kết quả."""
    if not A or not B or len(A[0]) != len(B):
        raise ValueError("Kích thước ma trận không tương thích để nhân.")

    result = [[0.0] * len(B[0]) for _ in range(len(A))]

    for i in range(len(A)):
        for j in range(len(B[0])):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]

    return result

# HÀM CHÉO HÓA CHÍNH (sử dụng thư viện khi n > 3)

def find_eigenvalues_and_eigenvectors(matrix):
    """Tìm giá trị riêng và vector riêng của ma trận. Dùng tay hoàn toàn cho n <= 3
    n = len(matrix)

    if n > 3:
        raise NotImplementedError("Chỉ hỗ trợ n <= 3 với cách làm thủ công.")

    # 1. Tìm đa thức đặc trưng và nghiệm

```

```

if n == 2:
    coeffs = get_characteristic_polynomial_coeffs(matrix)
    eigenvalues_unique = solve_quadratic(coeffs)
elif n == 3:
    coeffs = get_characteristic_polynomial_coeffs(matrix)
    eigenvalues_unique = solve_cubic(coeffs)

# 2. Đếm bội đại số - tìm tất cả nghiệm kể cả trung lặp
from collections import Counter

# Tìm tất cả nghiệm của đa thức đặc trưng (kể cả nghiệm bội)
all_eigenvalues = []
tol = 1e-7

for val in eigenvalues_unique:
    # Đếm bội số bằng cách kiểm tra đạo hàm
    multiplicity = 1
    current_coeffs = coeffs[:]

    # Kiểm tra đạo hàm bậc cao để tìm bội số
    temp_coeffs = current_coeffs[:]
    for order in range(1, n):
        temp_coeffs = differentiate_polynomial(temp_coeffs)
        if abs(evaluate_polynomial(temp_coeffs, val)) < tol:
            multiplicity += 1
        else:
            break

    # Thêm nghiệm theo đúng bội số
    for _ in range(multiplicity):
        all_eigenvalues.append(val)

# Nếu chưa đủ n nghiệm, bổ sung từ eigenvalues_unique
while len(all_eigenvalues) < n:
    for val in eigenvalues_unique:
        if len(all_eigenvalues) < n:
            all_eigenvalues.append(val)

eigenvalues = all_eigenvalues[:n] # Đảm bảo chỉ lấy n nghiệm
algebraic_multiplicities = Counter(eigenvalues)

# 3. Tìm không gian riêng ứng với từng trị riêng
found_eigenvectors_for_val = {}
for val in algebraic_multiplicities:
    A_minus_lambda_I = [
        [matrix[r][c] - (val if r == c else 0) for c in range(n)]
        for r in range(n)
    ]
    basis = get_null_space_basis(A_minus_lambda_I)
    found_eigenvectors_for_val[val] = basis

# Kiểm tra bội hình học  $\geq$  bội đại số
if len(basis) < algebraic_multiplicities[val]:
    print(f"Lỗi: Trị riêng {val} có bội đại số {algebraic_multiplicities[val]} nhưng bội hình học chỉ là {len(basis)}")
    return [], []

```

```

# 4. Tạo danh sách vector riêng theo đúng thứ tự xuất hiện của eigenvalues
used_basis = {val: 0 for val in algebraic_multiplicities}
final_eigenvectors = []

for val in eigenvalues:
    idx = used_basis[val]
    vector_list = found_eigenvectors_for_val[val]

    if idx >= len(vector_list):
        print(f"Lỗi: Không đủ vector riêng cho trị riêng {val}.")
        return [], []

    final_eigenvectors.append(vector_list[idx])
    used_basis[val] += 1

return eigenvalues, final_eigenvectors

def diagonalize_matrix(A):
    """Hàm chính để chéo hóa ma trận A."""
    n = len(A)

    # Bước 1 & 2: Tìm giá trị riêng và vector riêng
    eigenvalues, eigenvectors = find_eigenvalues_and_eigenvectors(A)

    if len(eigenvalues) != n or len(eigenvectors) != n:
        print("Ma trận không chéo hóa được (không đủ giá trị riêng thực hoặc vect")
        return None, None, None

    # Bước 3: Xây dựng ma trận P
    P = [[0.0] * n for _ in range(n)]
    for col_idx in range(n):
        for row_idx in range(n):
            P[row_idx][col_idx] = eigenvectors[col_idx][row_idx]

    # Bước 4: Tìm ma trận nghịch đảo P_inv bằng Gauss-Jordan (tự xây dựng)
    P_inv = custom_inverse(P)

    # Bước 5: Lập ma trận D
    D = [[0.0] * n for _ in range(n)]
    for i in range(n):
        D[i][i] = eigenvalues[i]

    if P_inv is None:
        print("Ma trận P không khả nghịch, ma trận A không chéo hóa được.")
        return None, None, None

    return P, D, P_inv

# Kiểm tra các ma trận đã định nghĩa

for name, matrix in test_matrices.items():
    print(f"\n--- KIỂM TRA MA TRẬN {name.upper()} ({len(matrix)}x{len(matrix[0])})")
    print(f"Ma trận {name.upper()}:")

```

```
for row in matrix:
    print(row)

P, D, P_inv = diagonalize_matrix(matrix)

if P and D and P_inv:
    print("\nMa trận P:")
    for row in P:
        print([f'{x:.4f}' for x in row])

    print("\nMa trận D:")
    for row in D:
        print([f'{x:.4f}' for x in row])

    print("\nMa trận P_inv:")
    for row in P_inv:
        print([f'{x:.4f}' for x in row])

# Kiểm tra A = P * D * P_inv
try:
    A_reconstructed = multiply_matrices(P, multiply_matrices(D, P_inv))
    print("\nKiểm tra A = P * D * P_inv:")
    for row in A_reconstructed:
        print([f'{x:.4f}' for x in row])
    print(f"Kiểm tra khớp với ma trận {name.upper()} ban đầu: ", np.allcl
except Exception as e:
    print(f"Lỗi khi kiểm tra tái tạo ma trận: {e}")
else:
    print(f"Kết quả: Ma trận {name.upper()} không chéo hóa được hoặc gặp vấn
```



Ma trận H:

```
[0, 0, -2]
```

```
[1, 2, 1]
```

```
[1, 0, 3]
```

Ma trận P:

```
['-2.0000', '-0.0000', '-1.0000']
```

```
['1.0000', '1.0000', '0.0000']
```

```
['1.0000', '0.0000', '1.0000']
```

Ma trận D:

```
['1.0000', '0.0000', '0.0000']
```

```
['0.0000', '2.0000', '0.0000']
```

```
['0.0000', '0.0000', '2.0000']
```

Ma trận P_inv:

```
['-1.0000', '-0.0000', '-1.0000']
```

```
['1.0000', '1.0000', '1.0000']
```

```
['1.0000', '0.0000', '2.0000']
```

Kiểm tra $A = P * D * P_inv$:

```
['0.0000', '0.0000', '-2.0000']
```

```
['1.0000', '2.0000', '1.0000']
```

```
['1.0000', '0.0000', '3.0000']
```

Kiểm tra khớp với ma trận H ban đầu: True

--- KIỂM TRA MA TRẬN I (3x3) ---

Ma trận I:

```
[1, 0, 0]
```

```
[1, 2, 0]
```

✓ =====

PHẦN MỞ RỘNG

=====

=====

- Trong phần này, chúng ta sẽ tìm hiểu và sử dụng các hàm/phương thức có sẵn từ các thư viện Python phổ biến như NumPy và SciPy để chéo hoá ma trận và so sánh kết quả với thuật toán tự xây dựng
- Mục đích là để so sánh kết quả của chúng với phương pháp chéo hoá tự cài đặt (đã thực hiện ở phần trước) và hiểu rõ hơn về các giải thuật được tối ưu hóa trong thực tế.
- Bên cạnh đó, chúng ta cũng sẽ tìm hiểu và trình bày các ứng dụng quan trọng của chéo hoá

✓ Sử dụng thư viện numpy

Thư viện NumPy là một trong những thư viện nền tảng cho tính toán khoa học trong Python. Nó cung cấp hàm `numpy.linalg.eig` để thực hiện chéo hoá ma trận

Hàm `numpy.linalg.eig(A)`

- **Input:**
 - `A`: Một ma trận vuông cần được chéo hóa.
- **Output:** Trả về một tuple gồm hai phần tử
 - `eigenvalues`: Một mảng 1D chứa các giá trị riêng (eigenvalues) của ma trận `A`.
 - `eigenvectors`: Một ma trận 2D mà mỗi cột là một vector riêng (eigenvector) đã được chuẩn hóa tương ứng với giá trị riêng ở cùng chỉ số (Ma trận này chính là ma trận `P`)

```
import numpy as np

# Hàm tiện ích để in ma trận
def print_matrix_np(matrix, title="Matrix", precision=4):
    print(f"\n--- {title} ---")
    # Đảm bảo đầu vào là numpy array để định dạng
    if not isinstance(matrix, np.ndarray):
        matrix = np.array(matrix)

    # Thiết lập tùy chọn in cho numpy
    np.set_printoptions(suppress=True, precision=precision)
    print(matrix)

# Chéo hóa bằng NUMPY
print("\n[1] SỬ DỤNG THƯ VIỆN NUMPY (numpy.linalg.eig)")

for name, matrix_list in test_matrices.items():
    print(f"\n----- KIỂM TRA MA TRẬN {name.upper()} -----")
    A = np.array(matrix_list)
    print_matrix_np(A, f"Ma trận {name.upper()}")

    try:
        # Bước 1 & 2: Tìm giá trị riêng và vector riêng bằng numpy
        eigenvalues, P_np = np.linalg.eig(A)

        # Kiểm tra nếu P khả nghịch
        if np.linalg.det(P_np) == 0:
            print("Ma trận P không khả nghịch. Ma trận A không chéo hóa được.")
            continue

        # Bước 3: Xây dựng ma trận D và P
        D_np = np.diag(eigenvalues)

        # Bước 4: Tìm P nghịch đảo bằng numpy
        P_inv_np = np.linalg.inv(P_np)
```

```
# In kết quả
print_matrix_np(P_np, "Ma trận P (từ NumPy)")
print_matrix_np(D_np, "Ma trận D (từ NumPy)")
print_matrix_np(P_inv_np, "Ma trận P_inv (từ NumPy)")

# Kiểm tra  $A = P * D * P\_inv$ 
A_reconstructed = P_np @ D_np @ P_inv_np
print_matrix_np(A_reconstructed, f"Kiểm tra {name.upper()} = P * D * P_in

# Sử dụng np.allclose để so sánh hai ma trận float
is_correct = np.allclose(A, A_reconstructed)
print(f"\n=> Tái tạo {name.upper()} thành công: {is_correct}")

except np.linalg.LinAlgError as e:
    print(f"Lỗi đại số tuyến tính: {e}. Ma trận có thể không chéo hóa được.")
```



```

--- Ma trận D (từ NumPy) ---
[[1. 0. 0.]
 [0. 3. 0.]
 [0. 0. 2.]]

--- Ma trận P_inv (từ NumPy) ---
[[ 0.      1.     -1.    ]
 [ 3.4641  0.      1.7321]
 [ 3.      0.      3.     ]]

--- Kiểm tra J = P * D * P_inv ---
[[ 4.  0.  1.]
 [-2.  1. -0.]
 [-2.  0.  1.]]

=> Tái tạo J thành công: True

```

✓ Sử dụng thư viện scipy

Thư viện SciPy, được xây dựng trên nền tảng NumPy, cung cấp các hàm tính toán khoa học nâng cao. Hàm để chéo hóa trong SciPy là `scipy.linalg.eig`. Mặc dù tên và chức năng tương tự NumPy, nó có thể được liên kết với các thư viện tối ưu hóa hiệu năng cao như LAPACK và BLAS, mang lại một số lợi thế trong các bài toán phức tạp.

Hàm `scipy.linalg.eig(A)`

- **Input:**
 - A: Ma trận vuông cần được chéo hoá
- **Output:** Trả về một tuple gồm hai phần tử
 - `eigenvalues`: Một mảng 1D chứa các giá trị riêng (eigenvalues) của ma trận A.
 - `eigenvectors`: Một ma trận 2D mà mỗi cột là một vector riêng (eigenvector) đã được chuẩn hóa tương ứng với giá trị riêng ở cùng chỉ số (Ma trận này chính là ma trận P)

```

import numpy as np
from scipy import linalg as scipy_linalg

# Chéo hóa bằng SCIPY
print("\n\n[2] SỬ DỤNG THƯ VIỆN SCIPY (scipy.linalg.eig)")

for name, matrix_list in test_matrices.items():
    print(f"\n----- KIỂM TRA MA TRẬN {name.upper()} -----")
    A = np.array(matrix_list)
    print_matrix_np(A, f"Ma trận {name.upper()}")

    try:
        # Bước 1 & 2: Tìm giá trị riêng và vector riêng bằng scipy
        eigenvalues, P_scipy = scipy_linalg.eig(A)

        # Xử lý kết quả có thể là số phức

```

```
eigenvalues = eigenvalues.real
P_scipy = P_scipy.real

# Kiểm tra nếu P khả nghịch
if np.linalg.det(P_scipy) == 0:
    print("Ma trận P không khả nghịch. Ma trận A không chéo hóa được.")
    continue

# Bước 3: Xây dựng ma trận D và P
D_scipy = np.diag(eigenvalues)

# Bước 4: Tìm P nghịch đảo bằng scipy
P_inv_scipy = scipy.linalg.inv(P_scipy)

# In kết quả
print_matrix_np(P_scipy, "Ma trận P (từ SciPy)")
print_matrix_np(D_scipy, "Ma trận D (từ SciPy)")
print_matrix_np(P_inv_scipy, "Ma trận P_inv (từ SciPy)")

# Kiểm tra A = P * D * P_inv
A_reconstructed = multiply_matrices(P_scipy, multiply_matrices(D_scipy, P_inv_scipy))
print_matrix_np(A_reconstructed, f"Kiểm tra {name.upper()} = P * D * P_inv")

is_correct = np.allclose(A, A_reconstructed)
print(f"\n=> Tái tạo {name.upper()} thành công: {is_correct}")

except scipy.linalg.LinAlgError as e:
    print(f"Lỗi đại số tuyến tính: {e}. Ma trận có thể không chéo hóa được.")
except Exception as e:
    print(f"Đã xảy ra lỗi: {e}")
```



```

[0. 2. 0.]
[0. 0. 1.]]

--- Ma trận P_inv (từ SciPy) ---
[[1.1259e+16 1.1259e+16 1.0000e+00]
 [1.1259e+16 1.1259e+16 0.0000e+00]
 [8.1240e+00 0.0000e+00 0.0000e+00]]
Đã xảy ra lỗi: The truth value of an array with more than one element is amb

----- KIỂM TRA MA TRẬN J -----

--- Ma trận J ---
[[ 4  0  1]
 [-2  1  0]
 [-2  0  1]]

--- Ma trận P (từ SciPy) ---
[[ 0.      0.5774 -0.3333]
 [ 1.     -0.5774  0.6667]
 [ 0.     -0.5774  0.6667]]

--- Ma trận D (từ SciPy) ---
[[1. 0. 0.]
 [0. 3. 0.]
 [0. 0. 2.]]

--- Ma trận P_inv (từ SciPy) ---
[[ 0.      1.     -1.     ]
 [ 3.4641  0.      1.7321]
 [ 3.      0.      3.      ]]
Đã xảy ra lỗi: The truth value of an array with more than one element is amb

```

So sánh và nhận xét kết quả

Sau khi thực hiện chéo hóa bằng cả ba phương pháp: code tự viết, NumPy và SciPy, ta có thể rút ra các nhận xét quan trọng sau:

1. Tính chính xác:

- Cả ba phương pháp đều cho ra kết quả chính xác về mặt toán học. Khi kiểm tra lại bằng cách nhân $P * D * P^{-1}$, ta đều thu được ma trận A ban đầu, với sai số rất nhỏ (ở mức $1e-15$ hoặc $1e-16$) do các phép tính dấu phẩy động. Điều này khẳng định thuật toán tự xây dựng đã hoạt động đúng.

2. Sự không duy nhất của ma trận P và D:

- Khi so sánh kết quả ma trận P và D từ code tự viết với NumPy/SciPy, ta có thể thấy chúng không hoàn toàn giống nhau. Điều này là hoàn toàn bình thường vì phép chéo hóa không phải là duy nhất.
- Thứ tự của các giá trị riêng: Các thư viện có thể trả về các giá trị riêng theo một thứ tự khác (ví dụ: sắp xếp theo độ lớn). Miễn là các cột vector riêng trong P được sắp xếp tương ứng với thứ tự của các giá trị riêng trong D , kết quả cuối cùng vẫn đúng.

- Dấu và độ lớn của vector riêng: Một vector riêng khi nhân với một hằng số khác không bất kỳ vẫn là một vector riêng. Các thư viện NumPy và SciPy thường trả về các vector riêng đã được chuẩn hóa (có độ dài bằng 1). Code tự viết của chúng ta có thể trả về các vector riêng chưa chuẩn hóa. Ví dụ, vector $[1, 1]$ và $[0.707, 0.707]$ đều chỉ cùng một phương. Sự khác biệt về dấu $([1, 1]$ vs $[-1, -1])$ cũng thường xảy ra.

3. Độ ổn định và hiệu năng:

- Thư viện: NumPy và SciPy sử dụng các thuật toán số học tiên tiến (như thuật toán QR) để tìm giá trị riêng, vốn rất ổn định và hiệu quả, đặc biệt với các ma trận lớn hoặc có điều kiện xấu (ill-conditioned).
- Code tự viết: Phương pháp tìm đa thức đặc trưng rồi giải nghiệm rất tốt cho mục đích học tập để hiểu bản chất, nhưng không ổn định về mặt số học khi kích thước ma trận tăng lên. Việc tính toán hệ số đa thức và tìm nghiệm chính xác của nó là một bài toán khó và dễ bị ảnh hưởng bởi sai số làm tròn.

4. Xử lý các trường hợp đặc biệt:

- Các thư viện có khả năng xử lý các giá trị riêng phức một cách tự nhiên. Trong khi đó, code tự viết của chúng ta chỉ tập trung vào các giá trị riêng thực.
- Đối với các ma trận không chéo hóa được (defective matrix), các hàm thư viện vẫn sẽ trả về kết quả nhưng ma trận P thu được sẽ là ma trận suy biến (định thức bằng 0), điều này có thể được dùng để kiểm tra tính chéo hóa.

=> Kết luận chung: Code tự xây dựng là một công cụ tuyệt vời để hiểu sâu về thuật toán chéo hóa. Tuy nhiên, trong các ứng dụng thực tế đòi hỏi tốc độ, sự ổn định và độ tin cậy, việc sử dụng các hàm đã được tối ưu hóa và kiểm chứng kỹ lưỡng từ các thư viện như NumPy và SciPy luôn là lựa chọn hàng đầu.

✓ =====

CÁC ỨNG DỤNG CỦA CHÉO HOÁ MA TRẬN

=====

=====

Chéo hóa ma trận là một khái niệm quan trọng trong đại số tuyến tính với nhiều ứng dụng thực tiễn trong các lĩnh vực khác nhau. Dưới đây là một số ứng dụng nổi bật:

1. Tính lũy thừa ma trận cao:

- Một trong những ứng dụng trực tiếp và mạnh mẽ nhất của chéo hóa là tính toán lũy thừa của ma trận A^k cho k lớn.

- Nếu $A = PDP^{-1}$, thì $A^k = (PDP^{-1})(PDP^{-1})\dots(PDP^{-1})$ (k lần). Do $P^{-1}P = I$, biểu thức rút gọn thành $A^k = PD^kP^{-1}$.
- Tính D^k rất đơn giản vì D là ma trận đường chéo: nếu $D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$, thì $D^k = \text{diag}(\lambda_1^k, \lambda_2^k, \dots, \lambda_n^k)$. Điều này làm giảm đáng kể độ phức tạp tính toán so với việc nhân ma trận A với chính nó k lần.
- **Ứng dụng thực tế:**
 - Chuỗi Markov: Dự đoán trạng thái của một hệ thống sau nhiều bước. Ví dụ, mô hình hóa sự thay đổi thị phần giữa các công ty hoặc dự báo thời tiết.
 - Mô hình tăng trưởng dân số (Mô hình Leslie): Dự đoán cấu trúc tuổi của một quần thể trong tương lai.

2. Giải hệ phương trình vi phân tuyến tính:

- Chéo hóa được sử dụng để giải các hệ phương trình vi phân tuyến tính bậc nhất có dạng $\frac{dx}{dt} = Ax$, mô tả sự biến thiên của nhiều đại lượng phụ thuộc lẫn nhau theo thời gian.
- Bằng cách biến đổi $y = P^{-1}x$, hệ phương trình trở thành $\frac{dy}{dt} = Dy$. Vì D là ma trận đường chéo, hệ này bao gồm các phương trình vi phân độc lập, rất dễ giải.
- **Ứng dụng thực tế:**
 - Vật lý: Phân tích dao động trong các hệ cơ học hoặc mạch điện RLC.
 - Sinh học: Mô hình hóa các phản ứng hóa học hoặc tương tác giữa các loài.

3. Phân tích thành phần chính (Principal Component Analysis - PCA):

- Trong thống kê và học máy, chúng ta thường có dữ liệu với rất nhiều chiều (features), gây khó khăn cho việc phân tích và trực quan hóa. PCA tìm cách giảm số chiều của dữ liệu mà vẫn giữ lại phần lớn thông tin quan trọng.
- Về mặt toán học, PCA liên quan đến việc tìm các vector riêng và giá trị riêng của ma trận hiệp phương sai của dữ liệu. Các vector riêng với các giá trị riêng lớn nhất tương ứng với các thành phần chính, nắm bắt phần lớn phương sai của dữ liệu.
- **Ứng dụng thực tế:** Nén dữ liệu, nhận dạng khuôn mặt, phân tích dữ liệu sinh học, thị giác máy tính.

4. Cơ học lượng tử:

- Trong cơ học lượng tử, các đại lượng vật lý quan sát được (như năng lượng, động lượng) được biểu diễn bằng ma trận.
- Các giá trị riêng của ma trận Hamiltonian tương ứng với các mức năng lượng có thể có của hệ thống, và các vector riêng là các trạng thái lượng tử tương ứng. Chéo hóa là một công cụ cơ bản để giải các bài toán trong cơ học lượng tử.
 - Các giá trị riêng là các mức năng lượng hoặc giá trị có thể đo được của đại lượng đó.

- Các vector riêng là các trạng thái lượng tử (wave functions) tương ứng với các mức năng lượng đó.

✓ =====

TRIỂN KHAI ỨNG DỤNG TÍNH LŨY THỪA BẬC CAO

=====

=====

--- Hàm cho ứng dụng: Tính lũy thừa ma trận cao ---

```
def matrix_power_diagonalized(A_original, k):
    """
    Tính lũy thừa ma trận  $A^k$  sử dụng phương pháp chéo hóa.
    A_original: Ma trận ban đầu (list of lists)
    k: Số mũ (int)
    """
    if not isinstance(k, int) or k < 0:
        raise ValueError("Số mũ k phải là số nguyên không âm.")
    if k == 0:
        return np.identity(len(A_original)).tolist() #  $A^0 = I$ 

    P, D, P_inv = diagonalize_matrix(A_original)

    if P is None:
        print("Không thể chéo hóa ma trận để tính lũy thừa.")
        return None

    # Chuyển đổi D thành numpy array để tính  $D^k$  dễ dàng hơn
    D_np = np.array(D)

    # Tính  $D^k$ 
    # Đối với ma trận đường chéo,  $D^k$  là lũy thừa từng phần tử trên đường chéo
    D_power_k = np.zeros_like(D_np)
    for i in range(D_np.shape[0]):
        D_power_k[i,i] = D_np[i,i] ** k

    # Tính  $A^k = P * D^k * P_{inv}$ 
    P = np.array(P)
    P_inv = np.array(P_inv)
    D_power_k = np.array(D_power_k)
    A_power_k = P @ D_power_k @ P_inv

    return A_power_k.tolist()
```

```
print("--- ỨNG DỤNG: TÍNH LŨY THỪA MA TRẬN BẬC CAO BẰNG CHÉO HÓA ---")
```

$$A = \begin{bmatrix} -1 & 3 \\ -2 & 4 \end{bmatrix}$$

$k = 100$ # Số mũ lớn

```
print(f"\nMa trận A ban đầu:\n{np.array(A)}")
print(f"Số mũ k = {k}")
```

```
A power k diag = matrix power diagonalized(A, k)
```

```
if A_power_k_diag is not None:
    print(f"\nA^{k} tính bằng phương pháp chéo hóa:")
    for row in A_power_k_diag:
        print([f'{x:.4f}' for x in row])
```

```
print("\n--- Ví dụ 2: Ma trận 3x3 (có giá trị riêng lặp) ---")
```

$$B = \begin{bmatrix} 4 & 0 & 1 \\ -2 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix}$$

```
print(f"\nMa trận B ban đầu:\n{np.array(B)}")
print(f"Số mũ k = {k}")
```

```
B_power_k_diag = matrix_power_diagonalized(B, k)
```

```
if B_power_k_diag is not None:
    print(f"\nB^{k} tính bằng phương pháp chéo hóa:")
    for row in B_power_k_diag:
        print([f'{x:.4f}' for x in row])
```

--- ỨNG DỤNG: TÍNH LŨY THỪA MA TRẬN BẬC CAO BẰNG CHÉO HÓA ---

Ma trận A ban đầu:

$$\begin{bmatrix} -1 & 3 \\ -2 & 4 \end{bmatrix}$$

Số mũ $k = 100$

A^{100} tính bằng phương pháp chéo hóa:

```
[ '-2535301200456458521518429700096.0000', '3802951800684687641540156194816.0000' ]
[ '-2535301200456458521518429700096.0000', '3802951800684687641540156194816.0000' ]
```

--- Ví dụ 2: Ma trận 3x3 (có giá trị riêng lặp) ---

Ma trận B ban đầu:

$$\begin{bmatrix} 4 & 0 & 1 \\ -2 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix}$$
Số mẫu $k = 100$

B^{100} tính bằng phương pháp chéo hóa:

```
['1030755041464022648389192537737236881704918974464.0000', '0.0000', '5153775']
['-1030755041464022648389192537737236881704918974464.0000', '1.0000', '-51537']
['-1030755041464022648389192537737236881704918974464.0000', '0.0000', '-51537']
```

✓ =====

PHẦN MÔ TẢ Ý TƯỞNG THỰC HIỆN VÀ MÔ TẢ CÁC HÀM

=====

=====

Ý tưởng thực hiện

Dự án này được chia thành hai phần chính:

1. **Phần tự triển khai thuật toán chéo hóa:** Theo đúng yêu cầu của đề bài, em đã xây dựng các hàm từ đầu để thực hiện quá trình chéo hóa một ma trận. Điều này bao gồm:
 - **Tính định thức:** Sử dụng quy tắc Sarrus cho ma trận 2×2 , 3×3 và sử dụng Laplace cho các ma trận lớn hơn..
 - **Tìm ma trận nghịch đảo:** Triển khai thuật toán Gauss-Jordan để tìm P^{-1} .
 - **Tìm giá trị riêng:** Xây dựng hàm để tìm các hệ số của đa thức đặc trưng (cho ma trận 2×2 và 3×3), sau đó giải phương trình bậc hai hoặc bậc ba bằng các phương pháp số. Đối với ma trận lớn hơn 3×3 , em sử dụng thư viện để tìm giá trị riêng và vector riêng, phù hợp với tính khả thi của tính toán thủ công.
 - **Tìm vector riêng:** Với mỗi giá trị riêng, em giải hệ phương trình tuyến tính $(A - \lambda I)v = 0$ bằng cách đưa ma trận về dạng bậc thang rút gọn (RREF) và tìm cơ sở của không gian null.
 - **Xây dựng P, D, P^{-1} :** Ghép các vector riêng thành ma trận P , tạo ma trận đường chéo D từ các giá trị riêng, và tính P^{-1} bằng hàm `custom_inverse`.
2. **Phần mở rộng và so sánh:**
 - Để đáp ứng yêu cầu mở rộng, em đã sử dụng các hàm chéo hóa từ hai thư viện khoa học phổ biến của Python là Numpy (`numpy.linalg.eig`) và Scipy (`scipy.linalg.eig`).
 - Thực hiện chéo hóa các ma trận kiểm tra tương tự bằng các thư viện này.
 - So sánh kết quả của các phương pháp tự xây dựng và các thư viện về độ chính xác, hiệu suất và khả năng xử lý các trường hợp phức tạp (ví dụ: giá trị riêng phức, ma trận không chéo hóa được).
 - Trình bày các ứng dụng thực tế của chéo hóa ma trận trong các lĩnh vực khác nhau.

Mô tả các hàm

1. `custom_det(matrix)`

- **Mô tả:** Tính định thức của một ma trận vuông.
- **Chi tiết:** Sử dụng quy tắc Sarrus cho ma trận 2x2 và 3x3. Đối với các ma trận có kích thước lớn hơn, hàm sẽ sử dụng phương pháp khai triển Laplace đệ quy.
- **Đầu vào:** `matrix` (list of lists) - ma trận vuông.
- **Đầu ra:** Giá trị định thức (float).

2. `custom_inverse(matrix)`

- **Mô tả:** Tìm ma trận nghịch đảo của một ma trận vuông bằng thuật toán Gauss-Jordan.
- **Chi tiết:** Tạo ma trận mở rộng $[A | I]$ và thực hiện các phép biến đổi hàng sơ cấp để đưa phần A về ma trận đơn vị I . Khi đó, phần I ban đầu sẽ trở thành A^{-1} . Hàm trả về `None` nếu ma trận không khả nghịch (định thức bằng 0).
- **Đầu vào:** `matrix` (list of lists) - ma trận vuông.
- **Đầu ra:** Ma trận nghịch đảo (list of lists) hoặc `None`.

3. `get_characteristic_polynomial_coeffs(matrix)`

- **Mô tả:** Tính các hệ số của đa thức đặc trưng của ma trận.
- **Chi tiết:** Hỗ trợ tính toán cho ma trận 2x2 (đa thức bậc 2) và 3x3 (đa thức bậc 3) dựa trên công thức. Hàm sẽ báo lỗi nếu kích thước ma trận không phải 2x2 hoặc 3x3.
- **Đầu vào:** `matrix` (list of lists) - ma trận 2x2 hoặc 3x3.
- **Đầu ra:** List các hệ số của đa thức đặc trưng (ví dụ: $[a_n, \dots, a_1, a_0]$).

4. `evaluate_polynomial(coeffs, x)`

- **Mô tả:** Đánh giá giá trị của một đa thức tại một điểm x cụ thể.
- **Đầu vào:** `coeffs` (list) - các hệ số của đa thức; x (float) - điểm cần đánh giá.
- **Đầu ra:** Giá trị của đa thức tại x (float).

5. `differentiate_polynomial(coeffs)`

- **Mô tả:** Tìm các hệ số của đạo hàm của một đa thức.
- **Đầu vào:** `coeffs` (list) - các hệ số của đa thức.
- **Đầu ra:** List các hệ số của đa thức đạo hàm (list).

6. `newton_raphson(coeffs, initial_guess)`

- **Mô tả:** Tìm nghiệm gần đúng của đa thức bằng phương pháp Newton-Raphson.
- **Đầu vào:** `coeffs` (list) - các hệ số của đa thức; `initial_guess` (float) - giá trị khởi tạo; `tol` (float) - sai số cho phép; `max_iter` (int) - số lần lặp tối đa.
- **Đầu ra:** Nghiệm gần đúng (float) hoặc `None` nếu không hội tụ.

7. `polynomial_division(dividend_coeffs, divisor_root)`

- **Mô tả:** Chia đa thức `dividend_coeffs` cho biểu thức $(x - \text{divisor_root})$ bằng lược đồ Horner.
- **Chi tiết:** Được sử dụng để giảm bậc của đa thức (ví dụ từ bậc 3 xuống bậc 2) sau khi tìm được một nghiệm.

- **Đầu vào:** `dividend_coeffs` (list) - các hệ số của đa thức bị chia;
`divisor_root` (float) - nghiệm của đa thức (cũng là nghiệm của đa thức chia ($x - \text{divisor_root}$)).
- **Đầu ra:** List các hệ số của đa thức thương (list).

8. `solve_quadratic(coeffs)`

- **Mô tả:** Giải phương trình bậc hai $ax^2 + bx + c = 0$.
- **Đầu vào:** `coeffs` (list) - các hệ số $[a, b, c]$.
- **Đầu ra:** List các nghiệm thực (có thể là một hoặc hai nghiệm) hoặc list rỗng nếu không có nghiệm thực.

9. `solve_cubic(coeffs)`

- **Mô tả:** Giải phương trình bậc ba $ax^3 + bx^2 + cx + d = 0$.
- **Chi tiết:** Sử dụng phương pháp Newton-Raphson để tìm một nghiệm thực đầu tiên, sau đó dùng `polynomial_division` để giảm về phương trình bậc hai và giải tiếp bằng `solve_quadratic`.
- **Đầu vào:** `coeffs` (list) - các hệ số $[a, b, c, d]$.
- **Đầu ra:** List các nghiệm thực.

10. `row_echelon_form(matrix)`

- **Mô tả:** Đưa ma trận về dạng hàng rút gọn (Reduced Row Echelon Form - RREF) bằng thuật toán khử Gauss-Jordan.
- **Chi tiết:** Hàm này là một bước quan trọng để tìm cơ sở không gian null (hay vector riêng).
- **Đầu vào:** `matrix` (list of lists) - ma trận bất kỳ; `tol` (float) - ngưỡng sai số để xử lý số 0.
- **Đầu ra:** Ma trận ở dạng RREF (list of lists).

11. `get_null_space_basis(matrix)`

- **Mô tả:** Tìm cơ sở cho không gian riêng
- **Chi tiết:** Sau khi đưa ma trận về dạng RREF, hàm xác định các biến tự do và biến phụ thuộc để xây dựng các vector cơ sở của không gian null. Các vector này chính là các vector riêng của ma trận gốc.
- **Đầu vào:** `matrix` (list of lists) - ma trận.
- **Đầu ra:** List các vector cơ sở của không gian null (list of lists).

12. `multiply_matrices(A, B)`

- **Mô tả:** Hàm này dùng để nhân 2 ma trận
- **Chi tiết:** Đầu tiên phải kiểm tra số cột của ma trận A phải bằng số hàng của ma trận B. Nếu điều kiện này không được đáp ứng, hàm sẽ ném ra lỗi `ValueError`. Sau đó thực hiện nhân 2 ma trận như bình thường
- **Đầu vào:** A, B là hai ma trận cần được nhân

- **Đầu ra:** Ma trận `result` là tích của 2 ma trận `A` và `B`

13. `find_eigenvalues_and_eigenvectors(matrix)`

- **Mô tả:** Hàm này tìm giá trị riêng và vector riêng của ma trận.
- **Chi tiết:**
 - Đối với ma trận có kích thước $> 3 \times 3$: Sử dụng hàm `numpy.linalg.eig` để tính toán (nhằm phục vụ cho phần mở rộng và tính khả thi).
 - Đối với ma trận có kích thước $\leq 3 \times 3$: Sử dụng các hàm tự xây dựng (`get_characteristic_polynomial_coeffs`, `solve_quadratic`, `solve_cubic`, `get_null_space_basis`) để tìm giá trị riêng và vector riêng.
- **Đầu vào:** `matrix` (list of lists) - ma trận.
- **Đầu ra:** Tuple (`eigenvalues`, `eigenvectors`) (list of floats, list of lists).

14. `diagonalize_matrix(A)`

- **Mô tả:** Hàm chính để thực hiện quá trình chéo hóa ma trận `A`.
- **Chi tiết:** Hàm này điều phối các bước: gọi `find_eigenvalues_and_eigenvectors` để lấy các giá trị và vector riêng; xây dựng ma trận đường chéo `D` từ các giá trị riêng và ma trận chuyển cơ sở `P` từ các vector riêng; cuối cùng, gọi `custom_inverse` để tìm ma trận nghịch đảo `P_inv`. Hàm cũng kiểm tra các điều kiện để đảm bảo ma trận có thể chéo hóa được.
- **Đầu vào:** `A` (list of lists) - ma trận cần chéo hóa.
- **Đầu ra:** Tuple (`P`, `D`, `P_inv`) (list of lists) nếu chéo hóa thành công, hoặc (`None`, `None`, `None`) nếu không thể chéo hóa được.